

CSCI 5380 – Network Virtualization and Orchestration

Lab 4 Amazon Web Services (AWS)

University of Colorado Boulder
Department of Computer Science
Network Engineering

Professor Levi Perigo, Ph.D.

Summary

Amazon Web Services (AWS) is a cloud-computing platform offered by Amazon. Cloud technology and tools are popular for network engineering, and the skills learned in this lab will enhance students' resumes in a desirable skillset in the current market. The AWS tools and skills used throughout this lab include EC2, security policies, S3 buckets, SNS, and Cloud Watch. Although these services could be managed using the AWS Management Console, AWS also offers a powerful tool (Software Development Kit) known as Boto3. This gives the power to manage the above-mentioned services for a large number of resources using simple Python scripts integrated with the Boto module.

Objectives

1. Learn about Amazon Web Services (AWS) tools
2. Learn how to deploy EC2 server instances
3. Learn how to deploy applications on those instances
4. Learn how to create security policies/firewall rules
5. Learn how to backup configurations into S3 buckets
6. Learn how to setup and use Simple Notification Service (SNS)
7. Learn how to setup Cloud Watch monitoring system
8. Manage AWS resources using Boto3 (Python based SDK)

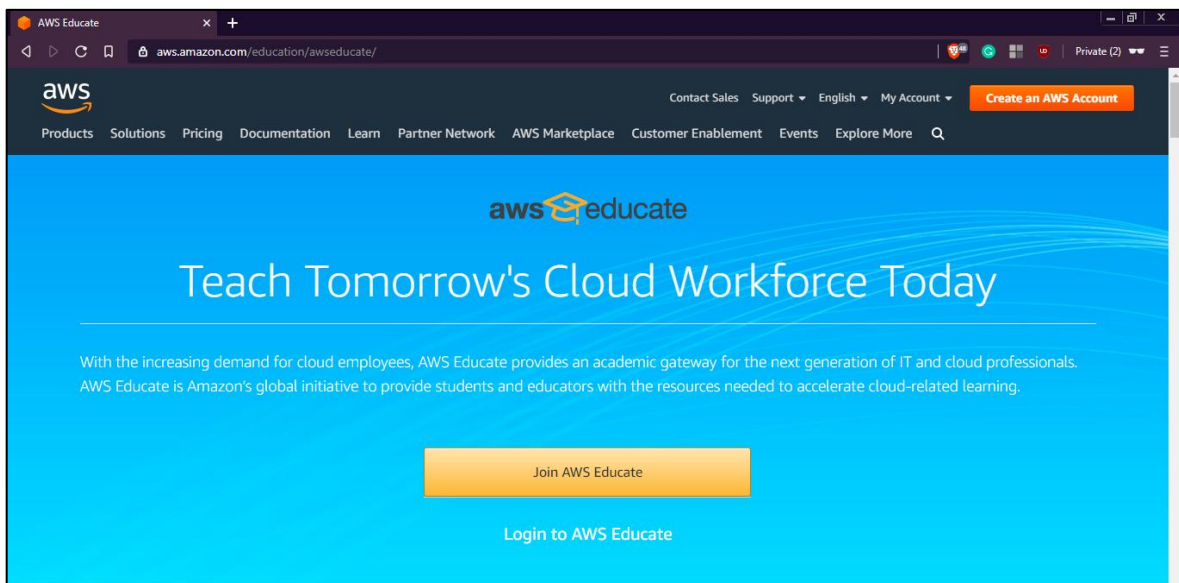
Part 1

Objective 1.1 – Create an AWS account

In order to use Amazon Web Services, you must first create a user account. Amazon provides one year of free usage (limited resources).

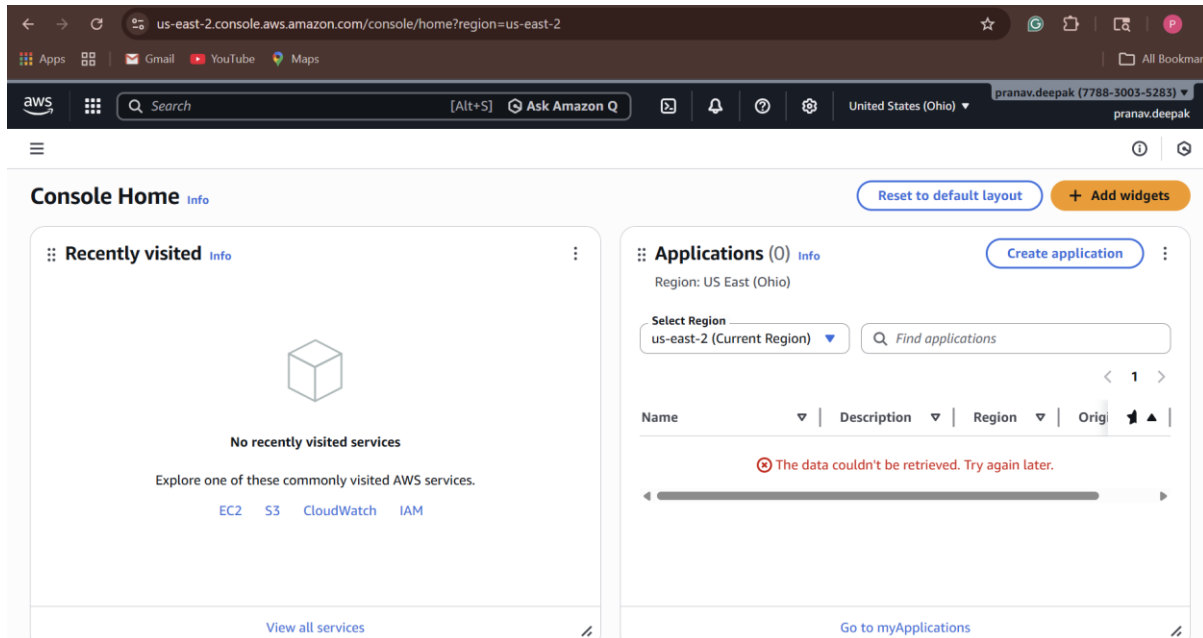
1. Create an AWS Educate account at

<https://aws.amazon.com/education/awseducate/> to get AWS promotional credits. You can also use your amazon.com credentials to log in, but please ensure you create a student account to get the AWS credits.



[NOTE: You might need to provide your debit/credit card number for future billing]

2. After creating the account, you will be directed to AWS management console which presents the user with a number of AWS tools and services. Provide a screenshot of your management console. [1 point]



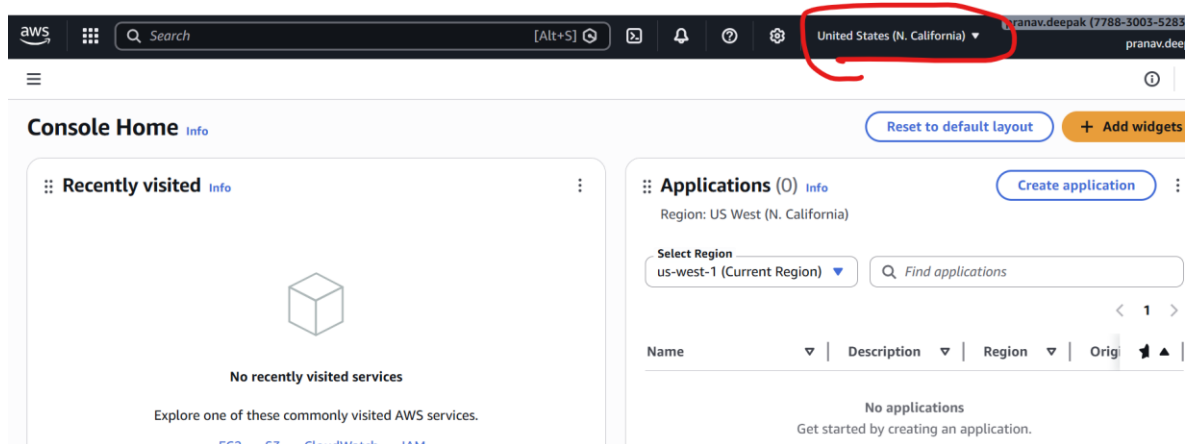
Objective 1.2 – Tweaking AWS settings

We shall now look at tweaking the AWS management settings to make efficient use of these resources.

1. We do not want Amazon to charge us for either using their premium services (Multi-Core CPU's, Terabytes of data, CDN services, etc.) or breaching their ONE-year free usage tier.

To prevent extra charges on your account please shut down and remove all instances at the end of the lab.

To ensure you get the most efficient performance, we need to change our region to any of the US West regions (N. California/Oregon). You can do this from the AWS management console. Provide a screenshot of the region you selected [1 point].

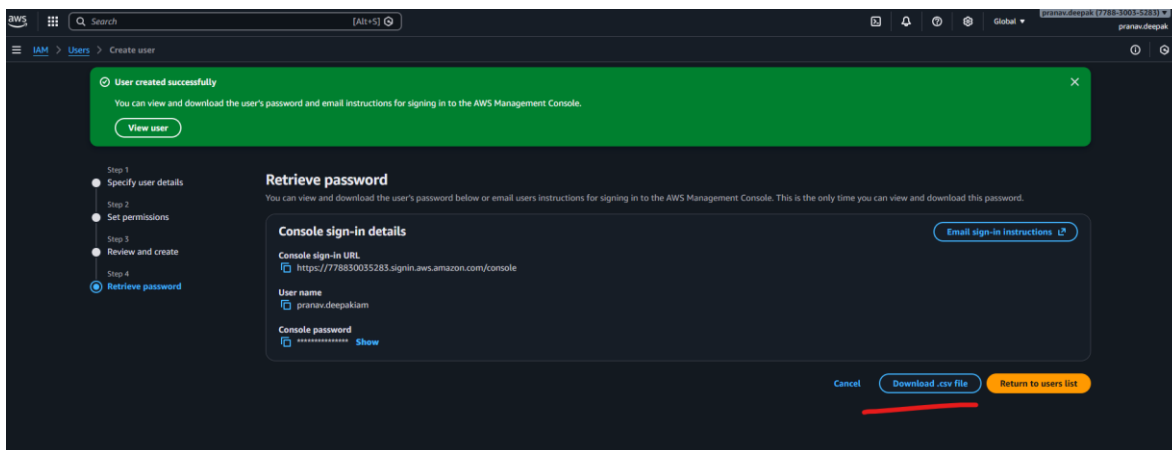


2. Set up security credentials for your account:

<https://aws.amazon.com/premiumsupport/knowledge-center/create-access-key/>

(Please ensure that you download and save the keys on your machine, as it will be required for further objectives).

1. First Create an IAM user. Just search IAM on the management console search bar and start configuring your new IAM user.
2. Once you create the IAM user, you will see this screen. Make sure to download the csv file that contains the IAM user creds



Part 2

Objective 2.1 – Deploying EC2 Instances

1. What is EC2? [5 points]

Ans] EC2 is one of AWS's many tools. It stands for Amazon Elastic Compute cloud. It allows the user to kind of rent virtual servers/instances from AWS' cloud infrastructure. The user can choose their preferred compute configuration and will be billed accordingly.

2. Briefly explain the below types of EC2 instances. **[15 points]**

- **Spot instances**

Spot instances are EC2 instances that use spare compute available with EC2 infrastructure. These are very cheap to use, up to 90% less than dedicated instances. These are great if short activity must be carried out by spinning up some instances. These can be a great money-saving deal. The drawback is that EC2 can reclaim these instances whenever it needs back with just a 2 minute warning. Hence, for long workloads or workloads with little or no recovery/checkpoints should not be carried out on spot instances.

- **Reserved instances**

Reserved instances make you commit for 1 or 3 years for a lower cost. These are stable instances and are ideal for long and steady workloads without any interruptions.

- **Dedicated hosts**

In dedicated hosts an entire server is dedicated to the user. The user has full control over hardware, sockets, cores and licenses. This is great for having full control over your hardware and also to impose your own SLA's and compliance

- **Dedicated instances**

Dedicated instances is very similar to dedicated hosts. The difference lies in the control of the hardware. In dedicated instances the user is given dedicated hardware but the control of the hardware lies with AWS. Hence, less control than dedicated hosts but more than shared instances

- **Elastic GPUs**

These are kind of addons or attachments to your EC2 instances that enable GPU acceleration for better graphics rendering and visualization.

Applications revolve around 3D graphics, video processing, and ML

To deploy an EC2 instance, click on Services> EC2 > Launch Instance.

Step 1: Search for ubuntu and select Ubuntu Server 16.04 LTS 64-bit (x86) image.

[Or you can select an image of your choice.]

Step 2: t2.micro (Free tier eligible)

NOTE: Selecting anything else will result in OS costs.

Step 3: Deploy **TWO** instances.

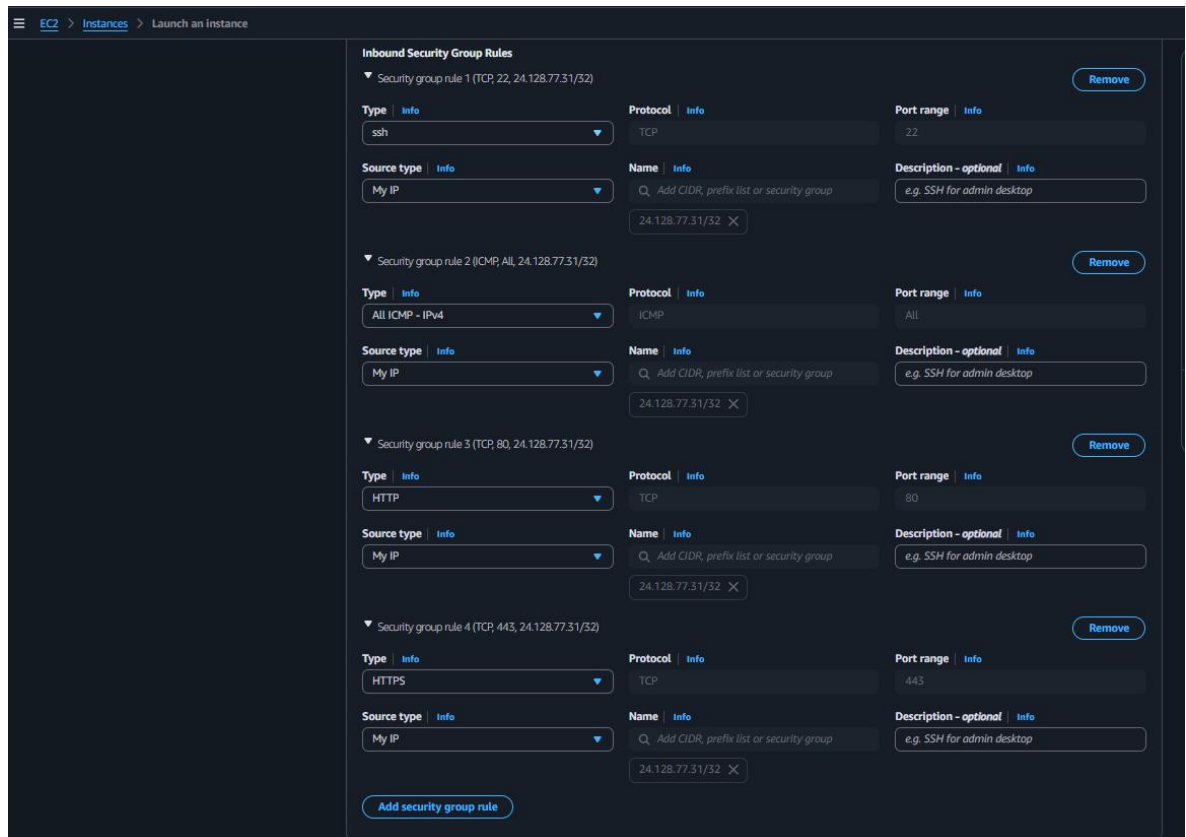
Step 4: Add storage (8 GB is fine, can add more later)

Step 5: Add name as NVO-Lab

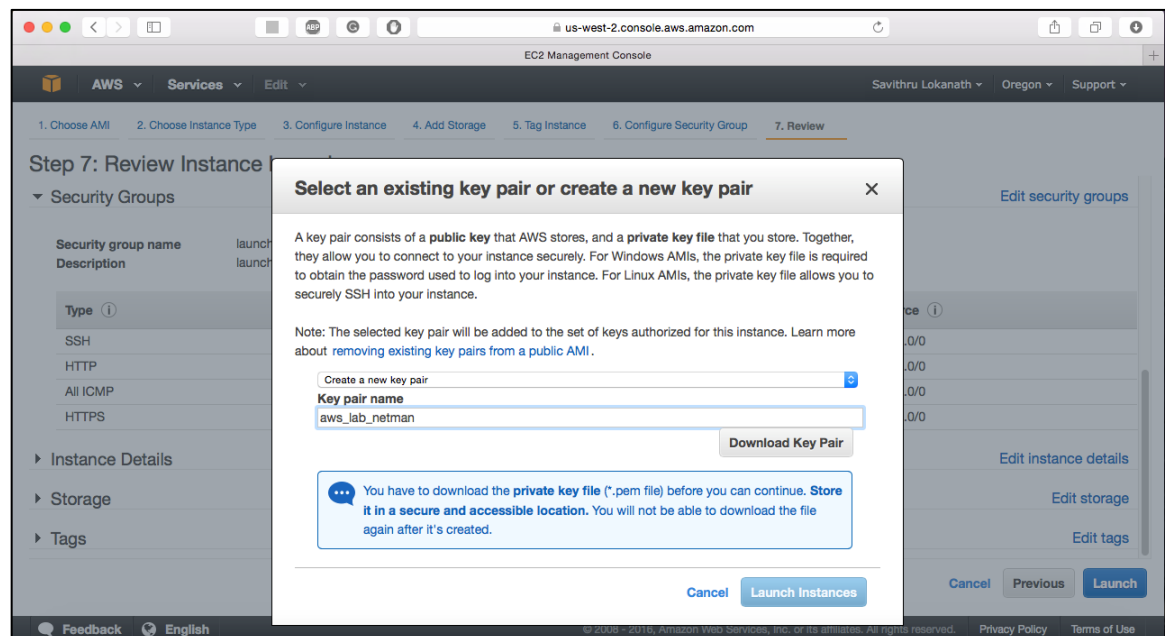
This uniquely identifies the instance and will come handy in a clustered environment. You can define something like a “webserver” if you are deploying a webserver or “appserver” for an application server, etc.

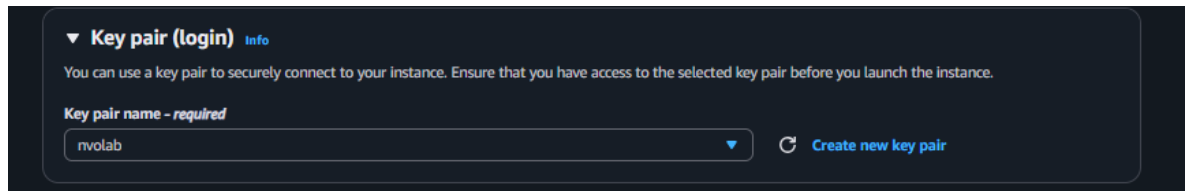
The screenshot displays the AWS 'Launch Instance' wizard. The 'Application and OS Images' section is active, showing a search bar and a 'Quick Start' section with icons for Amazon Linux, Ubuntu, Windows, Red Hat, SUSE Linux, and Debian. Below this, the 'Amazon Machine Image (AMI)' section lists 'Ubuntu Server 24.04 LTS (HVM), SSD Volume Type' as the selected AMI, with details like 'ami-0290e60ec230db1e4' and 'Free tier eligible'. The 'Description' section provides more details about the AMI. The 'Instance type' section shows 't3.micro' as the selected instance type, with details like 'Family: t3', '2 vCPU', and '1 GiB Memory'. The 'Summary' section on the right provides a overview of the configuration: 'Number of instances: 2', 'Software image (AMI): Canonical, Ubuntu, 24.04, amd64...', 'Virtual server type (instance type): t3.micro', 'Firewall (security group): New security group', and 'Storage (volumes): 1 volume(s) - 8 GiB'. The 'Launch Instance' button is highlighted in orange.

Step 6: Create a new security group and select the source to be your IP address for SSH rule. Add security rules allowing ICMP, HTTP and HTTPS traffic from your IP address.



Step 7: Review the configuration parameters before launching the instance. Create a new key-pair and click on download. [**NOTE:** If you miss this step, you will have to repeat the whole process again.] After the download completes, click on the “Launch” button.





3. Explain how an AMI is related to the instance. **[3 points]**

AMI(Amazon Machine Instance) is the image/OS for eg. Amazon linux, Ubuntu, Windows etc. that the instance is launched with. The user can choose the most appropriate OS and launch the instance. You can choose between different OS/AMIs when you try creating an instance.

4. What are the disadvantages of allowing SSH traffic into the server from anywhere?

Is this a security concern? **[5 points]**

Yes. It definitely is a security concern.

1. More prone to brute force attacks.
2. If my login credentials are exposed, the hacker can access my server from anywhere.
3. Ill use of server resources can increase bills and burn a hole in the user's pocket.

5. What are some best practices one should follow to secure the north-south and east-west traffic in the cloud? **[5 points]**

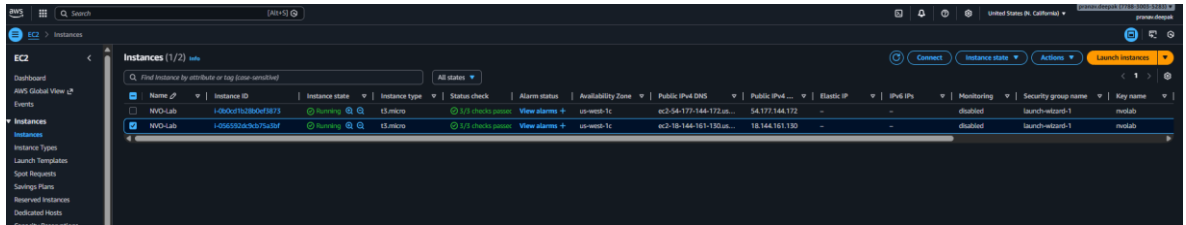
North South traffic:

1. Add firewall/security rules to restrict traffic and block DDoS attacks
2. Allow SSH access only from certain IP addresses. Always use secure methods to authenticate and use ssh like tokens.
3. Install load balancers to make sure no instances are overwhelmed.

East-West traffic:

1. Create segmentations in network to keep different traffic isolated.
2. Create firewall/security rules to restrict certain types of east-west traffic

6. Paste a screenshot showing the running instances. **[10 points]**



7. Select an instance and click on connect. Follow the instructions and SSH into one of the EC2 instances. Paste a screenshot showing the bash prompt. [Login for the Ubuntu instance is: ubuntu] [5 points]

```
ubuntu@ip-172-31-3-91: ~
Memory usage: 24%          Users logged in:      0
Swap usage:   0%          IPv4 address for ens5: 172.31.3.91

* Ubuntu Pro delivers the most comprehensive open source security and
compliance features.

https://ubuntu.com/aws/pro

Expanded Security Maintenance for Applications is not enabled.
0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

Last login: Sun Feb  8 21:17:11 2026 from 24.128.77.31
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@ip-172-31-3-91:~$
```

8. What IP address did you SSH to? And what is the IP address on the interface of the instance? Explain the flow of traffic from your laptop to the EC2 instance. [5 points]

I did an SSH into a public IP: **18.144.161.130**

The ip address on the interface is **172.31.3.91/20** which is a private IP address.

When I SSH from my laptop to the public IP address of my instance, it first goes through NAT on my ISP end and then the traffic is routed to AWS's internet gateway. The IGW maintains a table that maps the public IP address to my

instance's IP address. So traffic to and from my instance gets translated by the AWS's IGW

9. Create another user in your instance with the username being your name and enable SSH with password for just your user. Explain how you achieved this. [3 points]

1. First create a key pair on your local machine/computer: I used windows powershell and then entered the bash mode
2. `ssh-keygen -t rsa -b 4096 -f ~/.ssh/pranav`
3. `chmod 600 ~/.ssh/pranav`
4. `cat ~/.ssh/pranav.pub`. Copy the contents of the pranav.pub file. We will have to add this on the EC2 instance
5. Now go to your EC2 instance and then add a new user: `sudo adduser pranav`
6. `sudo usermod -aG sudo pranav`
7. `sudo su - pranav`
8. `mkdir ~/.ssh chmod 700 ~/.ssh touch ~/.ssh/authorized_keys chmod 600 ~/.ssh/authorized_keys`
9. `nano ~/.ssh/authorized_keys`
10. `chmod 755 /home/pranav`

Now we can ssh into the instance using our new user with the command:

`ssh -i ~/.ssh/pranav pranav@<EC2-Public-IP>`

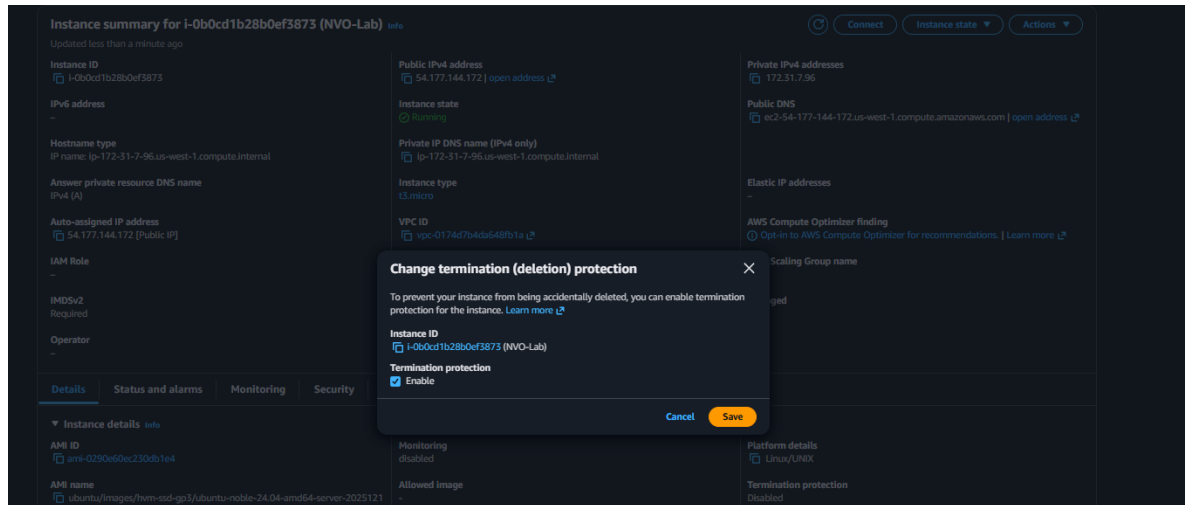
```
pranav@ip-172-31-3-91: ~  
prana@pranav:~/.ssh$ ssh -i ~/.ssh/pranav pranav@18.144.161.130  
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-1018-aws x86_64)  
  
* Documentation:  https://help.ubuntu.com  
* Management:    https://landscape.canonical.com  
* Support:        https://ubuntu.com/pro  
  
System information as of Mon Feb  9 07:02:48 UTC 2026  
  
System load:  0.0           Temperature:      -273.1 C  
Usage of /:   27.9% of 6.71GB Processes:         112  
Memory usage: 25%          Users logged in:   0  
Swap usage:   0%           IPv4 address for ens5: 172.31.3.91  
  
* Ubuntu Pro delivers the most comprehensive open source security and  
  compliance features.  
  
  https://ubuntu.com/aws/pro  
  
Expanded Security Maintenance for Applications is not enabled.  
  
0 updates can be applied immediately.  
  
Enable ESM Apps to receive additional future security updates.  
See https://ubuntu.com/esm or run: sudo pro status  
  
The list of available updates is more than a week old.  
To check for new updates run: sudo apt update  
  
The programs included with the Ubuntu system are free software;  
the exact distribution terms for each program are described in the  
individual files in /usr/share/doc/*/copyright.  
  
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by  
applicable law.  
  
pranav@ip-172-31-3-91:~$
```

10. What is the difference between stopping and terminating an instance? [2 points]

Stopping an instance means just shutting down the instance and retaining all the configs in memory. If at all we bring up the instance again, it will have all the configs that were there before stopping.

Terminating an instance permanently deleting that instance all together. Nothing will be salvageable post terminating an instance

11. Protect the second instance that you created from unauthorized termination. How did you achieve this? [2 points]



Click on your instance>>click on actions>>instance settings>>change termination protection>>Enable

Objective 2.2 – Deploying an application on your instance

Install Apache web server on one of the instances. Create an index.html file in the /var/www/html/ directory which displays your name and ‘NVO Lab’. You can use the below sample.

```
<HTML>
<HEAD>
<TITLE>NVO Lab</TITLE>
</HEAD>
<BODY>
<H2>Your_Name</H2>
</BODY>
</HTML>
```

How do you access this webpage hosted on the instance from your laptop? Paste relevant screenshots. Can you access it using an IP address or DNS name or both? [20 points]



Just type in the public IP address of the instance on the browser's URL.

DNS could work but I will have to get a domain name and link it with this IP. Also, DNS propagation would take a lot of time.

Part 3

Objective 3.1 – Deploying S3 Backups

NOTE: It is preferred to use the course VM provided to complete this objective.

1. What is S3? [5 points]

S3 is a storage service/solution offered by AWS. Unlike a traditional computer hard drive, it does not have a directory where it stores files inside folders. It has an infinite storage bank where every user can create a bucket for themselves. Each bucket is uniquely named so that they can be called when required using their unique name.

The files that the user uploads in these buckets are called objects. So whenever a user wants to call an object(file) the URL would be `s3.amazonaws.com/<user-bucket-name>/<file-name>`

2. You shall now backup some of our router configurations to the cloud. Before you proceed, ensure you have configuration files of your routers stored in a separate folder (Eg. `/home/nvo/routerConfigs`)

[**NOTE:** If you do not have configuration files present on your system, feel free to use other files (images, files, text,etc.)]

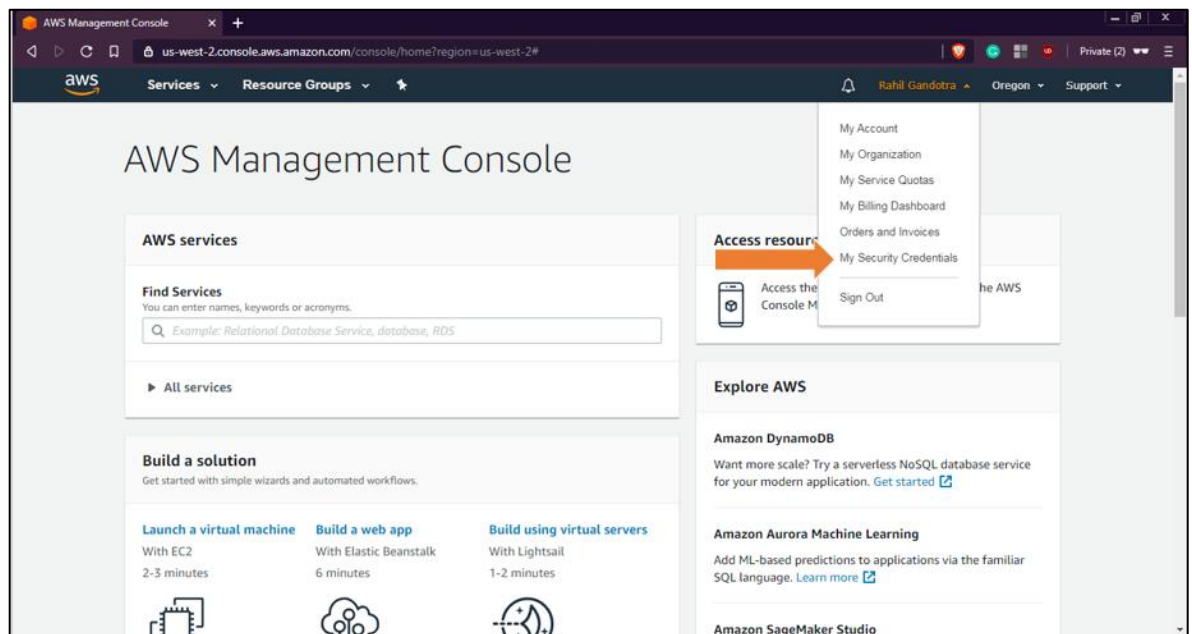
3. Update your VM and install S3 command-line utility

```
(VM)# sudo apt-get install s3cmd -y
```

4. Configure S3 parameters by entering your AWS access/secret credentials and enter the encryption password (Your choice) when prompted. Save the settings.

```
(VM)# s3cmd -configure
```

You can create/find an AWS access/secret key on your “username” > Security Credentials > Continue > Access Keys > Create/Use



```
HTTP Proxy server port: 0

Test access with supplied credentials? [Y/n] Y
Please wait, attempting to list all buckets...
Success. Your access key and secret key worked fine :-)

Now verifying that encryption works...
Not configured. Never mind.

Save settings? [y/N] y
Configuration saved to '/home/student/.s3cfg'
student@csci5380-vm1-prpr4002:~$
```

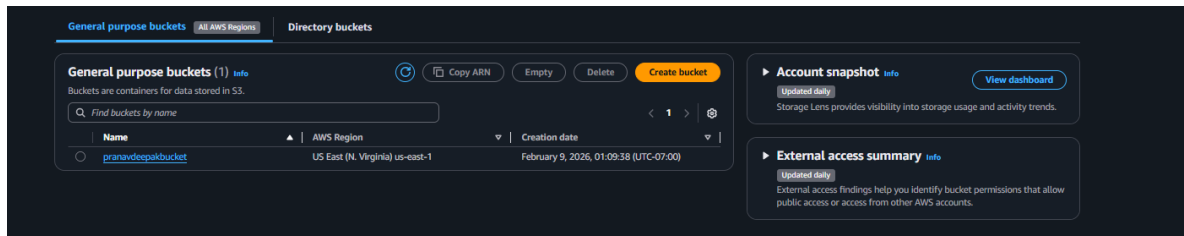
5. Create an S3 bucket

```
(VM)# s3cmd mb s3://<S3_BUCKET_NAME_CREATED>
```

```
student@csci5380-vm1-prpr4002:~$ s3cmd mb s3://pranavdeepakbucket
Bucket 's3://pranavdeepakbucket/' created
student@csci5380-vm1-prpr4002:~$
```

Confirm by issuing “**s3cmd ls**”. You can also check using the AWS Management Console. S3 is located under Storage and Content Delivery.

```
student@csci5380-vm1-prpr4002:~$ s3cmd ls
2026-02-09 08:09 s3://pranavdeepakbucket
```

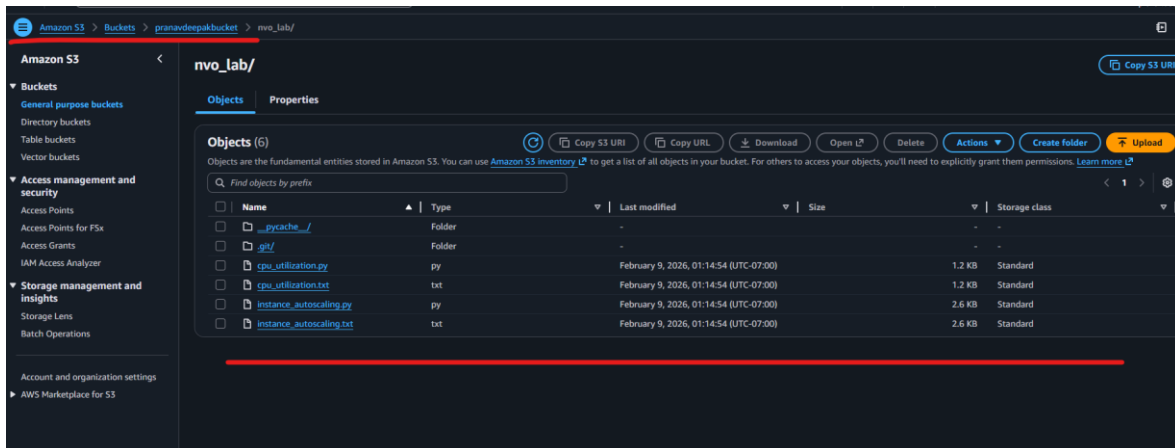


6. Push the configuration folder into the bucket

```
(VM)# s3cmd put <PATH_TO_LOCAL_CONFIG_FOLDER> s3://<
S3_BUCKET_NAME_CREATED>
```

Verify that the files are updated. Paste a screenshot showing the same. **[10 points]**

```
student@csci5380-vm1-prpr4002:~/nvo_lab$ pwd
/home/student/nvo_lab
student@csci5380-vm1-prpr4002:~/nvo_lab$ s3cmd put /home/student/nvo_lab s3://pranavdeepakbucket
ERROR: Parameter problem: Use --recursive to upload a directory: /home/student/nvo_lab
student@csci5380-vm1-prpr4002:~/nvo_lab$ s3cmd put /home/student/nvo_lab s3://pranavdeepakbucket --recursive
upload: '/home/student/nvo_lab/.git/COMMIT_EDITMSG' -> 's3://pranavdeepakbucket/nvo_lab/.git/COMMIT_EDITMSG' [1 of 36]
15 of 15 100% in 0s 78.36 B/s done
upload: '/home/student/nvo_lab/.git/HEAD' -> 's3://pranavdeepakbucket/nvo_lab/.git/HEAD' [2 of 36]
23 of 23 100% in 0s 353.81 B/s done
upload: '/home/student/nvo_lab/.git/config' -> 's3://pranavdeepakbucket/nvo_lab/.git/config' [3 of 36]
258 of 258 100% in 0s 3.90 KB/s done
upload: '/home/student/nvo_lab/.git/description' -> 's3://pranavdeepakbucket/nvo_lab/.git/description' [4 of 36]
73 of 73 100% in 0s 1123.84 B/s done
upload: '/home/student/nvo_lab/.git/hooks/applypatch-msg.sample' -> 's3://pranavdeepakbucket/nvo_lab/.git/hooks/applypatch-msg.sample' [5 of 36]
478 of 478 100% in 0s 6.99 KB/s done
upload: '/home/student/nvo_lab/.git/hooks/commit-msg.sample' -> 's3://pranavdeepakbucket/nvo_lab/.git/hooks/commit-msg.sample' [6 of 36]
896 of 896 100% in 0s 13.10 KB/s done
upload: '/home/student/nvo_lab/.git/hooks/fsmonitor-watchman.sample' -> 's3://pranavdeepakbucket/nvo_lab/.git/hooks/fsmonitor-watchman.sample' [7 of 36]
4726 of 4726 100% in 0s 66.71 KB/s done
upload: '/home/student/nvo_lab/.git/hooks/post-update.sample' -> 's3://pranavdeepakbucket/nvo_lab/.git/hooks/post-update.sample' [8 of 36]
189 of 189 100% in 0s 2.83 KB/s done
upload: '/home/student/nvo_lab/.git/hooks/pre-applypatch.sample' -> 's3://pranavdeepakbucket/nvo_lab/.git/hooks/pre-applypatch.sample' [9 of 36]
424 of 424 100% in 0s 6.21 KB/s done
upload: '/home/student/nvo_lab/.git/hooks/pre-commit.sample' -> 's3://pranavdeepakbucket/nvo_lab/.git/hooks/pre-commit.sample' [10 of 36]
1643 of 1643 100% in 0s 22.76 KB/s done
upload: '/home/student/nvo_lab/.git/hooks/pre-merge-commit.sample' -> 's3://pranavdeepakbucket/nvo_lab/.git/hooks/pre-merge-commit.sample' [11 of 36]
416 of 416 100% in 0s 5.80 KB/s done
upload: '/home/student/nvo_lab/.git/hooks/pre-push.sample' -> 's3://pranavdeepakbucket/nvo_lab/.git/hooks/pre-push.sample' [12 of 36]
1374 of 1374 100% in 0s 18.90 KB/s done
upload: '/home/student/nvo_lab/.git/hooks/pre-rebase.sample' -> 's3://pranavdeepakbucket/nvo_lab/.git/hooks/pre-rebase.sample' [13 of 36]
4898 of 4898 100% in 0s 66.75 KB/s done
upload: '/home/student/nvo_lab/.git/hooks/pre-receive.sample' -> 's3://pranavdeepakbucket/nvo_lab/.git/hooks/pre-receive.sample' [14 of 36]
544 of 544 100% in 0s 8.18 KB/s done
upload: '/home/student/nvo_lab/.git/hooks/prepare-commit-msg.sample' -> 's3://pranavdeepakbucket/nvo_lab/.git/hooks/prepare-commit-msg.sample' [15 of 36]
1492 of 1492 100% in 0s 22.31 KB/s done
```

7. What is another way of transferring data to your Amazon S3 bucket? [2 points]

You can go to the S3 console on AWS and then click on the upload button to upload files that you want.

8. Create a **cronjob** to sync every night. Paste a screenshot of the **cronjob** created. [5 points]

```
student@csci5380-vm1-prpr4002: ~/nvo_lab
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
m h dom mon dow  command
1 * * * s3cmd put /home/student/nvo_lab s3://pranavdeepakbucket --recursive
```

9. What is Amazon Glacier and how is it different from S3? [3 points]

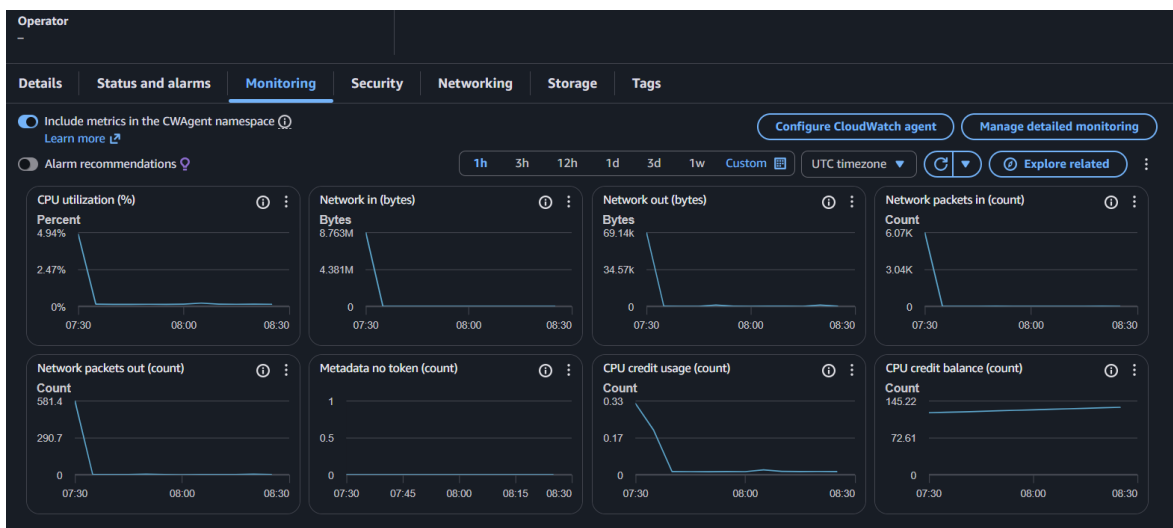
Amazon Glacier is another storage solution but this is used for archiving data rather than storing data that would be used frequently. The main applications of glacier would be to

store data such as archival records or files that do not need to be access for a very long period of time but need to be fetched if at all there is a need to. Glacier is much cheaper than S3 but the data retrieval time is very slow. Depending on the tier of glacier plan selected, data retrieval can take milliseconds(most expensive) to hours(cheapest).

Part 4

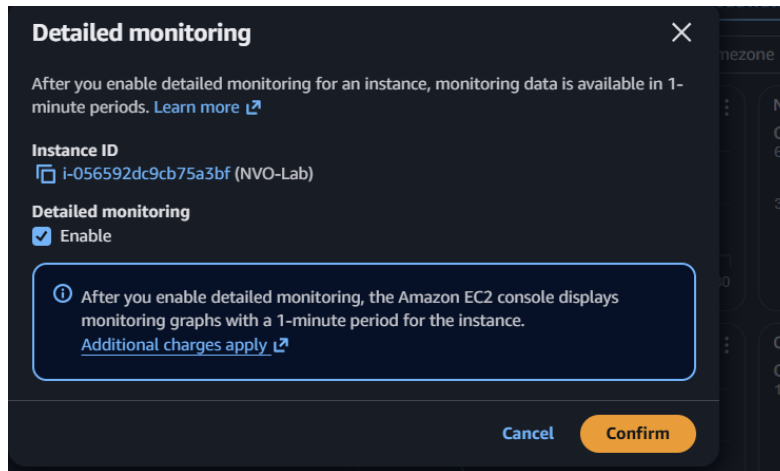
Objective 4.1 – Monitoring using CloudWatch

1. Click on any of the running EC2 instances and check the monitoring tab for EC2 metrics. What are the key metrics that you see and why is this important for an organization? [5 points]



We see CPU utilization, CPU credit usage and packets that came in and out of the instance. These are great to access the current usage of the instance and trends in the processes and traffic handled by that instance. It also helps the user to optimize and plan how resources need to be allocated to the instance for best efficiency

2. Enable detail monitoring for CloudWatch metrics. What is the difference between Basic monitoring and Detailed monitoring? [5 points]

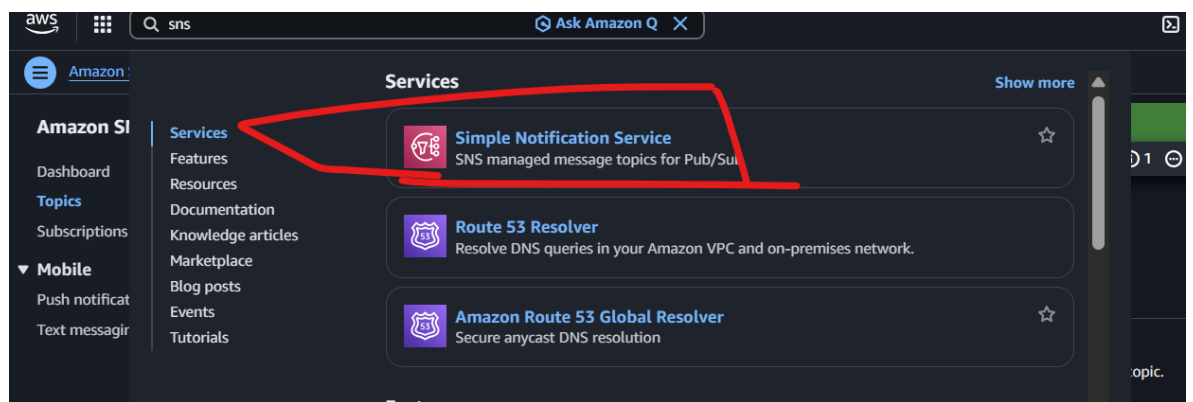


As mentioned with detailed monitoring, the monitoring graphs will display metrics with a 1-minute period for the instance

3. Create a new **CloudWatch alarm** to monitor **average CPU utilization**. The alarm should take effect when average CPU utilization is greater than a user-defined threshold. Alarm should send an **E-Mail** to the recipients entered during the alarm configuration. Paste a screenshot of the alarm created and the Email that you received. [10 points]

[**NOTE:** For simplicity, enter the threshold to be less than 1%. Or you can use the [Linux stress tool](#) to generate artificial stress on your instance to increase its CPU utilization.]

1. Create an SNS topic first with an email subscription



Amazon SNS > Topics > Create topic

New Feature
Amazon SNS now supports High Throughput FIFO topics. [Learn more](#)

Create topic

Details

Type [Info](#)

Topic type cannot be modified after topic is created

☐ FIFO (first-in, first-out)

- Strictly-preserved message ordering
- Exactly-once message delivery
- Subscription protocols: SQS

☒ Standard

- Best-effort message ordering
- At-least once message delivery
- Subscription protocols: SQS, Lambda, Data Firehose, HTTP, SMS, email, mobile application endpoints

Name

Maximum 256 characters. Can include alphanumeric characters, hyphens (-) and underscores (_).

Display name - optional [Info](#)

To use this topic with SMS subscriptions, enter a display name. Only the first 10 characters are displayed in an SMS message.

Maximum 100 characters.

Encryption - optional

Amazon SNS provides in-transit encryption by default. Enabling server-side encryption adds at-rest encryption to your topic.

CPU_alert Edit Delete Publish message

Details

Name

CPU_alert

Topic owner

778830035283

ARN

arn:aws:sns:us-west-1:778830035283:CPU_alert

Display name

-

Type

Standard

Subscriptions | Access policy | Data protection policy | Delivery policy (HTTP/S) | Delivery status logging | Encryption | Tags | Integrations

Subscriptions (0) Edit Delete Request confirmation Confirm subscription Create subscription

ID	Endpoint	Status	Protocol
No subscriptions found You don't have any subscriptions to this topic. Create subscription			

Amazon SNS > Subscriptions > Create subscription

New Feature
Amazon SNS now supports High Throughput FIFO topics. [Learn more](#)

Create subscription

Details

Topic ARN

Protocol

The type of endpoint to subscribe

Email

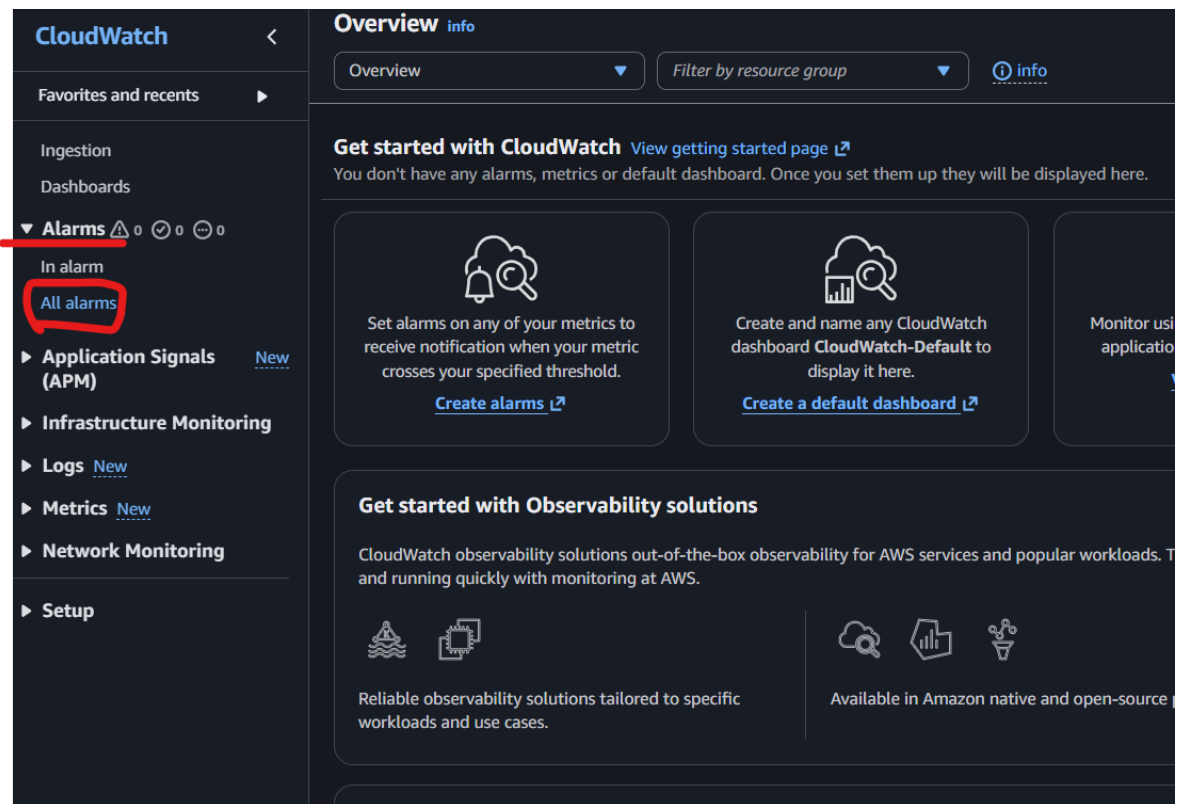
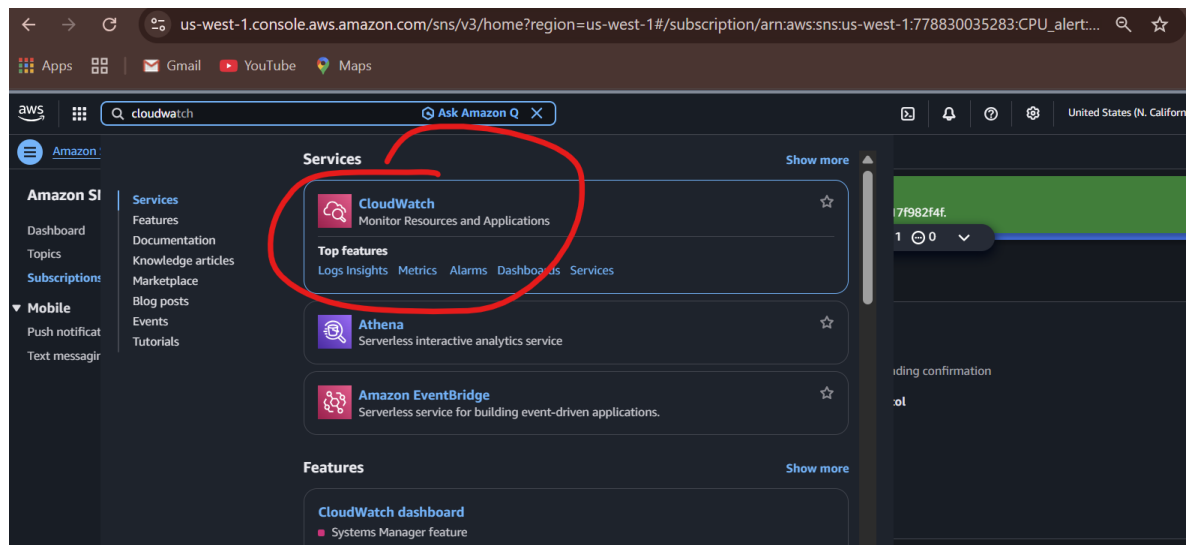
Endpoint

An email address that can receive notifications from Amazon SNS.

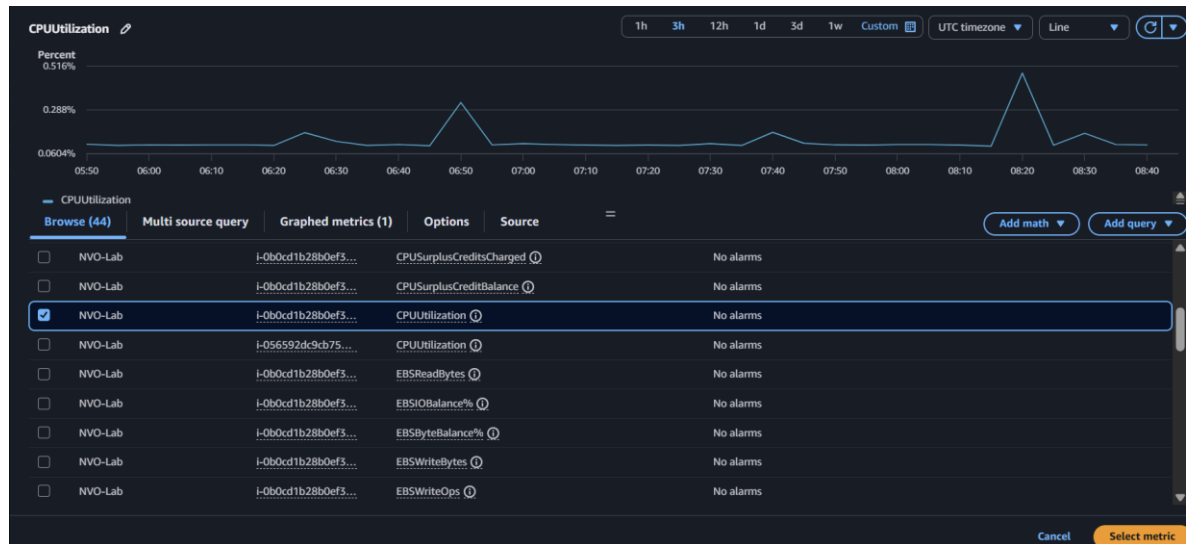
[After your subscription is created, you must confirm it.](#)

A confirmation email will be sent to the email address. Just confirm it once

2. Now configure cloudwatch



Click on create alarm>>select metric>>EC2>>pre instance>>check on CPU utilization for ONE instance at a time



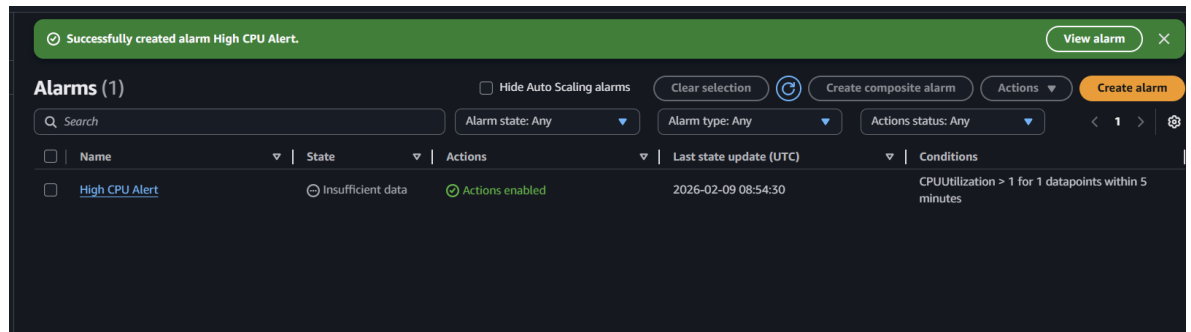
Set the conditions on the next window

The screenshot shows the 'Conditions' configuration window in the AWS CloudWatch console. The 'Static' threshold type is selected, and the condition is set to 'Greater' than the threshold of 1. The 'Additional configuration' section is expanded, showing the 'Alarm state trigger' and 'Send a notification to the following SNS topic' options.

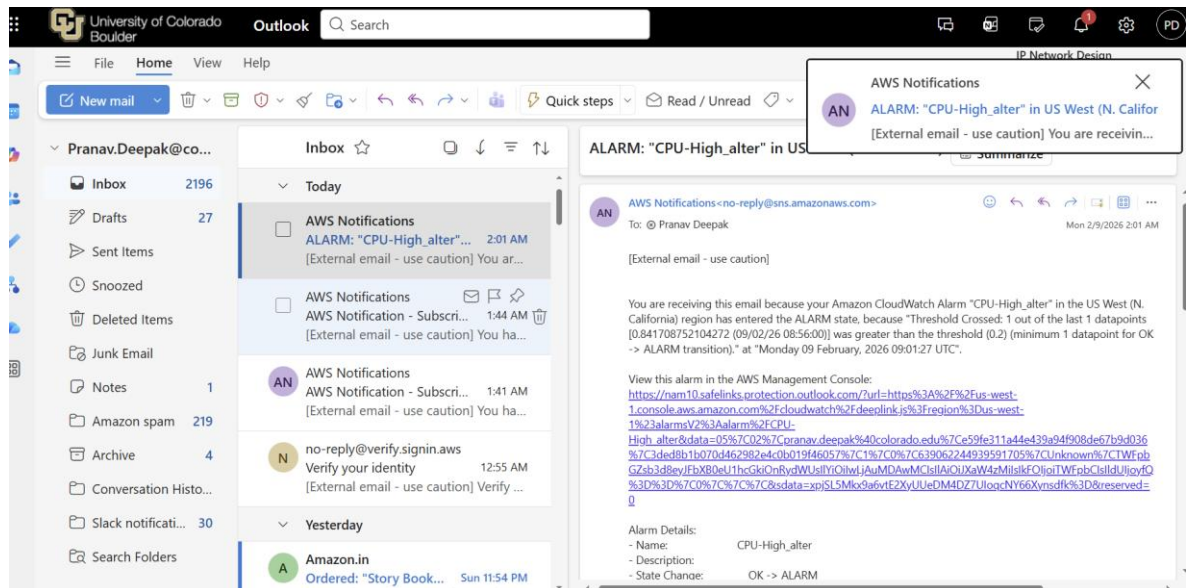
Select your SNS topic that you configured previously

The screenshot shows the 'Configure actions' window in the AWS CloudWatch console. The 'In alarm' state trigger is selected, and the 'Send a notification to the following SNS topic' option is chosen. The topic 'CPU_alert' is selected, and the 'Add notification' button is visible.

Give a name to the alarm>>preview and create



Now whenever the CPU utilization of the instance crosses the threshold we will get an email notification



4. Which is the service used by CloudWatch to send out E-Mail notifications? [5 points]

AS mentioned above the service used is called as SNS(Simple Notification Service)

Part 5

Objective 5.1 Setting up BOTO3 for AWS resource automation

1. Install Boto 3 on your machine using “sudo -H pip install boto3” to download the required packages.
2. Before we start using Boto, it is mandatory to setup the necessary authentication to the AWS management console. In order to do this, we would need to download

the AWS CLI and put in the AWS Access Keys which we have already downloaded in objective 1.3.

3. You may install the awscli for ease of authentication using “sudo -H pip install awscli” command. Further steps to achieve this are found in the below mentioned link:

<https://boto3.amazonaws.com/v1/documentation/api/latest/guide/quickstart.html#installation>.

After installing awscli, we need to set the access key and the secret key for seamless code integration.

Run the below command on your local machine:

```
aws configure
```

This will ask you to enter key details and the region name that you want your instances or any other service to spin on

4. If you opt to not set up one-time authentication with AWS CLI, ensure you add `Var_name = Session (aws_access_key_id = ' ', aws_secret_access_key = ' ', region_name = ' ')` in your python code for further objectives.

Objective 5.2 Controlling EC2 using Boto3

1. Write a python script to launch two new EC2 instances, stop one of the instances, and then fetch the details of all instances using the Boto3 module. Sample output:
[Instance Id] [Instance_type] [Instance_ip_address] “Running/Stopped”
[Hint: refer <https://boto3.readthedocs.org/en/latest/guide/migrationec2.html>]
2. Submit the .py file that you created to accomplish this objective, screenshots showing the new instances created on the console, and the instance details as specified in the sample output. **[30 points]**


```

• student@csci5380-vm1-prpr4002:~/nvo_lab$ /bin/python3 /home/student/nvo_lab/boto3_aws_create_ec2.py
Launching instances...
Created instances: ['i-0472ac4ae47003425', 'i-06ee696e27966b186']
Stopping instance i-0472ac4ae47003425...
Fetching updated details...

```

Instance Id	Type	IP Address	Status
i-0be7a10e63bd4f704	t3.micro	N/A	terminated
i-00de141518f721086	t3.micro	N/A	terminated
i-0472ac4ae47003425	t3.micro	3.101.28.47	stopping
i-06ee696e27966b186	t3.micro	13.56.197.144	running

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone
	i-0be7a10e63bd4f704	Terminated	t3.micro	-	View alarms +	us-west-1a
	i-00de141518f721086	Terminated	t3.micro	-	View alarms +	us-west-1a
	i-0472ac4ae47003425	Stopping	t3.micro	-	View alarms +	us-west-1a
	i-06ee696e27966b186	Running	t3.micro	Initializing	View alarms +	us-west-1a
NVO-Lab	i-0b0cd1b28b0ef3873	Running	t3.micro	5/3 checks passed	View alarms +	us-west-1c
NVO-Lab	i-056592dc9cb75a3bf	Terminated	t3.micro	-	View alarms +	us-west-1c

Objective 5.3 Fetching Cloudwatch metrics using Boto3

1. Write a python script to create a new AWS session using access keys (refer objective 1), create a cloudwatch session, and fetch the following metrics for one running EC2 instance over a specific time period (at least 30 minutes):

Status_Check, CPU_Utilization, Network_In and Network_Out. Sample output:

Instance ID: <value>

Status Check: <value>

CPU Utilization: <value>

Network In: <value>

Network Out: <value>

[Hint:

https://boto3.amazonaws.com/v1/documentation/api/latest/reference/services/cloudwatch.html#CloudWatch.Client.get_metric_statistics]

2. Submit the .py file that you created to accomplish this objective and screenshots of the details as specified in the sample output. [20 points]

```

• student@csci5380-vm1-prpr4002:~/nvo_lab$ /bin/python3 /home/student/nvo_lab/boto3_cloudwatch.py
Instance ID: i-06ee696e27966b186
Status Check: 0.00
CPU Utilization: 1.36%
Network In: 693.00 Bytes
Network Out: 697.00 Bytes
• student@csci5380-vm1-prpr4002:~/nvo_lab$

```

Objective 5.4 Spinning new instances based on CPU utilization

Write a python script that uses Boto3 to continuously fetch CPU utilization of two running EC2 instances. When a specific threshold is reached, the script should automatically shut the instances down, spin up identical new instances and send out an alert email to your email id. Submit the .py file that you created to accomplish this and relevant screenshots. **[20 points]**

This will use cloudwatch, ec2 and sns using boto3

```
student@csci5380-vm1-prpr4002:~/nvo_lab$ /bin/python3 /home/student/nvo_lab/boto3_cpu_usage_new_instance.py
Starting monitor for existing instances: ['i-0472ac4ae47003425', 'i-0b0cd1b28b0ef3873']
[16:06:56] i-0472ac4ae47003425: 0.12082972246294685%
⚠ High CPU (0.12082972246294685%) on i-0472ac4ae47003425. Terminating and replacing...
✅ Replacement Instance ID: i-08b969aa0c7be6912
[16:06:58] i-0b0cd1b28b0ef3873: 0.1041631251492985%
⚠ High CPU (0.1041631251492985%) on i-0b0cd1b28b0ef3873. Terminating and replacing...
```

Email notification

A screenshot of an email interface. At the top left, there's a button labeled "Existing EC2 Replaced". To its right is a button with a document icon and the word "Summarize". The email header shows it's from "AWS Notifications <no-reply@sns.amazonaws.com>" with a profile picture of a person named "AN". Below the header, the recipient is listed as "To: Pranav Deepak" and the time is "Mon 2/9/2026 4:07 PM". The main body of the email starts with "[External email - use caution]" followed by a paragraph stating: "Existing instance i-0b0cd1b28b0ef3873 was terminated due to 0.1041631251492985% CPU usage." This is followed by two dashes "--" and another paragraph starting with "If you wish to stop receiving notifications from this topic, please click or visit the link below to unsubscribe:" followed by a long URL. The bottom part of the email says "Please do not reply directly to this email. If you have any questions or comments regarding this email, please contact us at https://nam10.safelinks.protection.outlook.com/?url=https%3A%2F%2Faws.amazon.com%2Fsupport&data=05%7C02%7Cpranav.deepak%40colorado.edu%7C612a31b1c8eb460d898308de682fed57%7C3ded8b1b070d462982e4c0b019f46057%7C1%7C0%7C639062752238817464%7CUnknown%7CTWFpbGZsb3d8eyJFbXB0eU1hcGkiOnRydWUsIlYiOiilwLjAuMDAwMCIslIAlOiiXaW4zMlslkFOljoIjVWFBpCIsIdUljoyvQ%3D%3D%7C0%7C7C%7C7C%7C7C&sdata=rBFJvakhCo%2B93JDzROVztOr..."

Reflection:

1. Now that you have learnt the basics, what do you think are the most important reasons for an organization to use AWS? **[5 points]**

No need to buy your own infrastructure. Easy scaling of infrastructure on the click of a button. Comes with a lot of features that just works and integrates well. AWS has great API support. The boto3 wrapper function for python works great to integrate code with AWS features. No capex. Opex can vary how big your organization is

2. Suggest any other AWS modules that you would like to learn about. Why?

I would like to learn about the AWS load balancer module to understand how we can manage large traffic trying to accessing and interacting with the services that we are hosting

3. Suggest any other public cloud platforms that you would like to learn about. Why?

Microsoft Azure is another popular cloud platform which is being used a lot in industry. It would be great to try my hands on Azure as well!

Total Points _____ / 207